

FACAMP · TRATAMENTO E ARMAZENAMENTO DA INFORMAÇÃO

Torra & Terra

E-commerce de café especial em Flask e PostgreSQL.
Do levantamento de requisitos ao deploy em cloud.

PROFESSOR	Nivaldo T. Marcusso
APLICAÇÃO	https://nivaldo.felipefurlan.com.br
REPOSITÓRIO	github.com/hick12/projeto-nivaldo
BANCO	PostgreSQL 18.6 gerenciado · Railway
DATA	Setembro de 2026

1. O problema e a decisão de tema

Torrefações de café especial vendem por telefone, WhatsApp e planilha. O controle de estoque é manual e o pedido é anotado à mão, o que produz duas dores recorrentes: o mesmo lote vendido duas vezes porque duas vendas aconteceram em paralelo, e o pedido gravado pela metade — cabeçalho sem itens, ou total que não corresponde aos itens.

A pergunta central: como estruturar uma loja que liste os cafés, monte um carrinho e conclua o pedido *com integridade garantida pelo banco de dados*?

Por que café especial, e não um e-commerce genérico. Cada item do pedido carrega a **moagem** escolhida pelo cliente na hora da compra — grão, média ou fina. É um atributo que existe só no item, nunca no produto. É exatamente isso que faz `itens_pedido` ser uma **entidade** de verdade, e não uma mera tabela de ligação. O tema foi escolhido para tornar essa distinção concreta e demonstrável.

Atores

ATOR	OBJETIVO	NO MVP
Cliente	Encontrar cafés, escolher moagem, concluir a compra	SIM
Operação	Controlar estoque, pedidos e clientes	parcial — o estoque baixa sozinho, mas não há tela
Gestão	Relatórios de vendas e volume	FORA

Requisitos funcionais

ID	REQUISITO	STATUS
RF01	Catálogo lendo do banco, com filtro por região	ENTREGUE
RF02	Detalhe do produto: nota sensorial, torra, SCA	ENTREGUE
RF03	Carrinho com escolha de quantidade e moagem	ENTREGUE
RF04	Cadastro e login, senha em hash	ENTREGUE
RF05	Checkout transacional com baixa de estoque	ENTREGUE
RF06	Meus pedidos, com os itens de cada pedido	ENTREGUE

2. Modelo de dados

Cinco tabelas. As cardinalidades saem direto do domínio: um cliente faz vários pedidos; um pedido tem no mínimo um item; o mesmo café aparece em vários pedidos ao longo do tempo; cada café pertence a uma região.

```
clientes(id, nome, email UNIQUE, senha_hash, criado_em)
categorias(id, nome UNIQUE, regioao, descricao)
produtos(id, nome, descricao, preco, estoque, categoria_id,
          torra, nota_sensorial, pontuacao_sca, peso_g)
pedidos(id, cliente_id, status, total, criado_em)
itens_pedido(id, pedido_id, produto_id, quantidade,
             preco_unitario, moagem)
```

Por que não existe N:N no modelo

À primeira vista `pedidos` e `produtos` teriam um relacionamento N:N, resolvido por uma tabela de ligação. Não é o caso: `itens_pedido` tem **atributos próprios** — `quantidade`, `preco_unitario` e `moagem` — que não pertencem nem ao pedido nem ao produto. Isso a promove de tabela de ligação a **entidade associativa**. É a diferença entre "estes produtos estão neste pedido" e "esta quantidade deste café, moída assim, custou isto naquele dia".

Normalização

FORMA	COMO É ATENDIDA
1FN	Campos atômicos. Os itens do pedido são linhas em <code>itens_pedido</code> , não uma lista dentro de <code>pedidos</code> .
2FN	Na chave natural (<code>pedido_id</code> , <code>produto_id</code> , <code>moagem</code>), nenhum atributo depende de parte dela. <code>nome_do_produto</code> não está no item — é lido por JOIN.
3FN	<code>categorias</code> existe separada por isso. Se a descrição da região morasse em <code>produtos</code> , haveria <code>id</code> → <code>regiao</code> → <code>descricao</code> : dependência transitiva, com redundância e anomalias de inserção, atualização e exclusão.

A desnormalização deliberada. `itens_pedido.preco_unitario` duplica um dado que também está em `produtos.preco` — e isso não viola a normalização, porque os dois têm significados diferentes: um é "quanto custa hoje", o outro é "quanto custou naquela compra". É um fato histórico independente. Se o pedido lesse `produtos.preco`, o histórico se reescreveria a cada reajuste e o `total` gravado deixaria de bater com a soma dos itens.

3. As constraints vivem no banco

O `SQL/schema.sql` é DDL escrito à mão, executável no `psql . db.create_all()` não é chamado em lugar nenhum do projeto. A razão é direta: a validação do formulário protege a experiência do usuário; a constraint protege o **dado**. Se amanhã alguém inserir um registro por um script, um `INSERT` manual ou uma futura API, a regra continua valendo.

CHECK

11

FOREIGN KEY

4

PRIMARY KEY

5

UNIQUE

3

AS ONZE CHECK, COMO O BANCO AS GUARDA

```
ck_clientes_email_formato (POSITION('@'::text IN (email)) > 1)
ck_itens_moagem           moagem = ANY (ARRAY['GRAO','MEDIA','FINA'])
ck_itens_preco            (preco_unitario >= (0)::numeric)
ck_itens_quantidade       (quantidade > 0)
ck_pedidos_status        status = ANY (ARRAY['CRIADO','PAGO','ENVIADO','CANCELADO'])
ck_pedidos_total          (total >= (0)::numeric)
ck_produtos_estoque       (estoque >= 0)
ck_produtos_peso          (peso_g > 0)
ck_produtos_preco         (preco >= (0)::numeric)
ck_produtos_sca           (pontuacao_sca >= 80 AND pontuacao_sca <= 100)
ck_produtos_torra         torra = ANY (ARRAY['CLARA','MEDIA','ESCURA'])
```

A constraint mais didática é `ck_produtos_sca`. Café especial, pela definição da SCA, pontua 80 ou mais. Isso não é preferência de interface — é o que *define* o produto. É regra de negócio pura, e por isso mora no banco.

Dinheiro é NUMERIC, nunca FLOAT

Ponto flutuante binário não representa `0,10` de forma exata, e o erro se acumula ao somar os itens de um pedido. Todos os campos monetários são `NUMERIC(10,2)`.

Índices — apenas quatro, cada um justificado

ÍNDICE	CONSULTA QUE O MOTIVA
<code>idx_produtos_nome</code>	Ordenação do catálogo — a consulta mais executada da aplicação
<code>idx_produtos_categoria</code>	Filtro por região. O PostgreSQL não indexa coluna de FK automaticamente
<code>idx_pedidos_cliente</code>	Histórico do cliente — cresce sem limite ao longo do tempo
<code>idx_itens_pedido</code>	O JOIN que monta o detalhe de cada pedido

Índice acelera leitura, mas encarece escrita e ocupa disco. Criar "por precaução" é custo sem benefício.

4. A transação de checkout

O coração do trabalho. Tudo-ou-nada, em duas fases dentro de uma única transação.

```
BEGIN
|
| FASE 1 – travar e validar TODOS os itens
|   | SELECT ... FOR UPDATE (em ordem crescente de id)
|   |   | o produto ainda existe?
|   |   |   | o estoque cobre a quantidade?
|   |
| FASE 2 – gravar
|   | INSERT do pedido
|   | INSERT de cada item, com preco_unitario congelado
|   |   | UPDATE do estoque
|   |
| COMMIT . ROLLBACK em qualquer falha
```

Três decisões que valem explicar

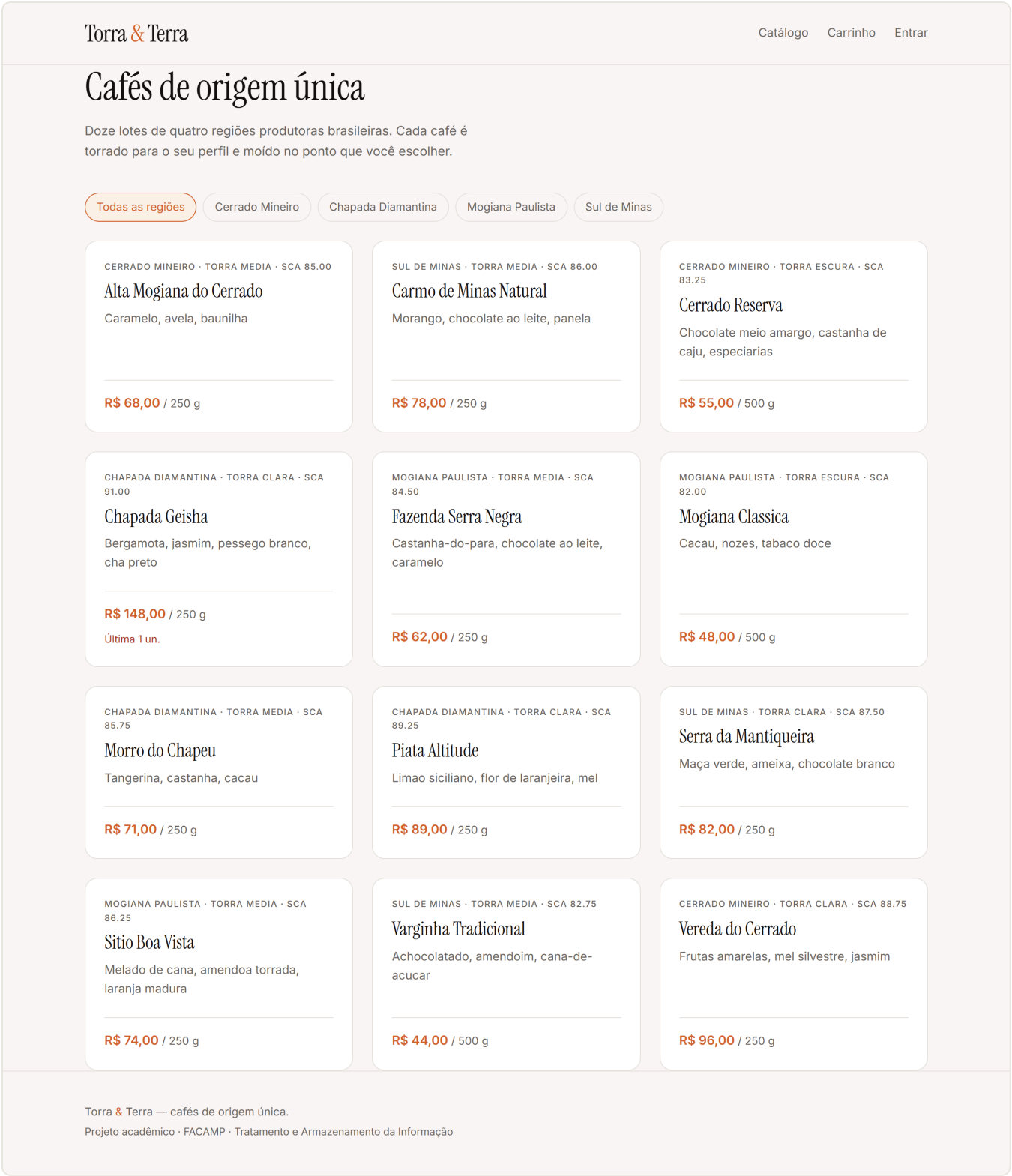
Nada é inserido antes da validação terminar. A fase 1 trava e confere todos os itens; só depois a fase 2 grava. O rollback continua obrigatório, mas o banco trabalha menos no caminho de erro.

Os produtos são travados em ordem crescente de `id`. Duas compras simultâneas que travassem os mesmos cafés em ordens opostas ficariam em *deadlock*, cada uma esperando o lock da outra. Ordenar elimina o ciclo antes que ele exista.

O `SELECT ... FOR UPDATE` é o que impede vender o mesmo lote duas vezes. Sem ele, duas compras simultâneas leem o mesmo estoque 1, ambas aprovam, e o café é vendido em dobro.

Três camadas de defesa, e vale saber que existem as três. 1) o campo de quantidade tem `max` igual ao estoque, e o navegador se recusa a enviar mais que isso; 2) a aplicação valida dentro da transação, com a linha já travada; 3) a constraint `estoque >= 0` é a última barreira, mesmo que as duas primeiras falhem.

5. A aplicação



Catálogo / Chapada Diamantina

CHAPADA DIAMANTINA · BAHIA

Chapada Geisha

Bergamota, jasmim, pessego branco, cha preto

Variedade Geisha adaptada a Bahia. Sete anos de lavoura para chegar a este perfil. O café mais premiado da casa.

TORRA	Clara
PONTUAÇÃO SCA	91.00
PESO	250 g
REGIÃO	Bahia

Altitudes acima de 1.000 m no semiárido baiano. Amplitude térmica alta e maturação lenta. Acidez cítrica marcante e aroma floral.

R\$ 148,00 / 250 g

Última 1 unidade

MOAGEM

Média — coador e prensa

A moagem é escolhida na compra, não no cadastro do café.

QUANTIDADE

1

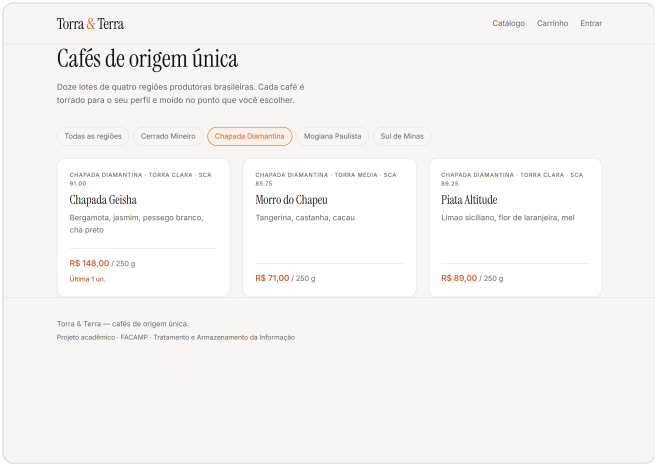
Adicionar ao carrinho

Torra & Terra — cafés de origem única.

Projeto acadêmico · FACAMP · Tratamento e Armazenamento da Informação

RF02 e RF03 — Detalhe e escolha da moagem. É aqui que o argumento da modelagem fica visível: a moagem é escolhida *na compra*, não no cadastro do café.

5. A aplicação (continuação)



Filtro por região — a consulta que justifica o `idx_produtos_categoria` .



Mobile-first: uma coluna no celular, três no desktop. CSS puro, sem framework.

Torra & Terra

Catálogo Carrinho Entrar

Criar conta

Nome

E-mail

Senha

Pelo menos 8 caracteres.

Criar conta

Já tem conta? Entrar

Torra & Terra — cafés de origem única.

Projeto acadêmico - FACAMP - Tratamento e Armazenamento da Informação

RF04 — cadastro. A senha vira hash scrypt antes de chegar ao banco.

Torra & Terra

Catálogo Carrinho Entrar

Entrar

E-mail

Senha

Entrar

Ainda não tem conta? [Criar conta](#)

Torra & Terra — cafés de origem única.

Projeto acadêmico - FACAMP - Tratamento e Armazenamento da Informação

Login. Credencial errada devolve mensagem única, sem revelar se o erro foi no e-mail ou na senha.

6. O cenário de erro

A demonstração mais importante do trabalho. O café *Chapada Geisha* nasce no `seed.sql` com **estoque 1**, de propósito.

Torra & Terra

Catálogo Carrinho 2 Meus pedidos Sair

Estoque insuficiente de Chapada Geisha: você pediu 2 e temos 1 em estoque.

Seu carrinho

Chapada Geisha

MOAGEM FINA · 2 × R\$ 148,00

Restam apenas 1 unidade deste café.

R\$ 296,00

Remover

TOTAL

R\$ 296,00

Finalizar compra

Continuar comprando

Esvaziar carrinho

Torra & Terra — cafés de origem única.

Projeto acadêmico · FACAMP · Tratamento e Armazenamento da Informação

Tentativa de comprar **2 unidades** de um café com estoque 1. A mensagem **nomeia o café** e informa o saldo real. O carrinho é preservado de propósito, para o cliente corrigir a quantidade sem remontar a compra.

A PROVA DE QUE NADA FOI GRAVADO

```
-- antes da tentativa
pedidos = 0      itens_pedido = 0
Chapada Geisha: estoque = 1

-- depois da tentativa que falhou
pedidos = 0      itens_pedido = 0      <- idênticos
Chapada Geisha: estoque = 1           <- intacto
pedidos órfãos (sem item) = 0
```

Rollback completo. Nem o cabeçalho do pedido sobrou. É a diferença entre uma transação de verdade e uma sequência de `INSERT` soltos.

7. A compra que dá certo

Torra & Terra

CatálogoCarrinhoMeus pedidosSair

Pedido #5 confirmado.

Meus pedidos

Pedido #5

02/09/2026 ÀS 22:58

CRIADO

R\$ 267,00

2 × Piata Altitude — MOAGEM GRAO

R\$ 178,00

1 × Piata Altitude — MOAGEM FINA

R\$ 89,00

Os valores acima são os preços da data da compra, congelados no pedido.

Pedido #4

02/09/2026 ÀS 22:56

CRIADO

R\$ 267,00

2 × Piata Altitude — MOAGEM GRAO

R\$ 178,00

1 × Piata Altitude — MOAGEM FINA

R\$ 89,00

Os valores acima são os preços da data da compra, congelados no pedido.

Pedido #1

26/08/2026 ÀS 23:34

CRIADO

R\$ 267,00

2 × Piata Altitude — MOAGEM GRAO

R\$ 178,00

1 × Piata Altitude — MOAGEM FINA

R\$ 89,00

Os valores acima são os preços da data da compra, congelados no pedido.

Torra & Terra — cafés de origem única.

Projeto acadêmico · FACAMP · Tratamento e Armazenamento da Informação

RF06 — Meus pedidos. Repare que cada pedido traz o mesmo café em duas linhas, porque a moagem difere.

O QUE O BANCO GUARDOU

```
-- o pedido
#5 | Demonstracao TAI | CRIADO | R$ 267,00 | 02/09/2026 22:58

-- os itens: duas linhas do mesmo café, moagens diferentes
pedido #5 | Piata Altitude | 2x | moagem GRAO | R$ 89,00 | subtotal R$ 178,00
pedido #5 | Piata Altitude | 1x | moagem FINA | R$ 89,00 | subtotal R$ 89,00

-- o total bate com a soma dos itens
divergências: 0

-- e o estoque desceu
Piata Altitude: 18 → 15
```

Esta é a prova concreta do argumento central. O mesmo produto, no mesmo pedido, em duas linhas, porque a moagem difere. Se moagem fosse coluna de produtos , isso seria impossível sem duplicar o cadastro do café — e o controle de estoque, que é do café e não da moagem, quebraria.

8. Testes e índices

```
platform win32 -- Python 3.11.9, pytest-8.3.4
collected 24 items

tests/test_checkout.py ..... [ 50%]
tests/test_seguranca.py ..... [100%]

===== 24 passed in 71.25s =====
```

Os testes rodam contra um **PostgreSQL real**, não SQLite: o `SELECT ... FOR UPDATE` e as constraints `CHECK` são justamente o que precisa ser testado, e o SQLite trata os dois de forma diferente. Testar contra outro banco daria confiança falsa.

CASO	O QUE PROVA
Compra normal	Pedido e itens gravados, estoque reduzido
Estoque insuficiente	Rollback total, nada gravado, mensagem nomeando o café
Produto inexistente	Pedido não é finalizado
Preço negativo · estoque negativo · SCA fora da faixa · moagem inválida	As constraints do banco recusam
Senha	Nunca gravada em texto puro
Mesmo café, duas moagens	<code>itens_pedido</code> é entidade
Preço congelado	Reajuste não altera pedido antigo
CSRF e cabeçalhos (12 testes)	As proteções de segurança seguem ativas

O que o EXPLAIN mostrou

CONSULTA	PLANO ESCOLHIDO
<code>WHERE categoria_id = 4</code>	Index Scan usando <code>idx_produtos_categoria</code>
<code>WHERE cliente_id = 2</code>	Bitmap Index Scan usando <code>idx_pedidos_cliente</code>
<code>ORDER BY nome</code> , sem filtro	Seq Scan + Sort

A explicação está na seletividade, e ela não é um problema. As duas primeiras consultas filtram poucas linhas de muitas — compensa consultar o índice. A terceira precisa de *todas* as linhas: usar o índice significaria ler o índice inteiro *e depois* a tabela inteira, mais trabalho e não menos. O planejador está certo nos três casos. O `idx_produtos_nome` passa a compensar quando o `Sort` não couber mais em memória.

9. Segurança

A aplicação foi submetida a uma varredura com **OWASP ZAP**, que apontou 11 alertas. Oito eram acionáveis e foram corrigidos.

ALERTA	CORREÇÃO
Ausência de tokens Anti-CSRF	Token na sessão, campo escondido nos 6 formulários, validado em <code>before_request</code>
Cookie sem SameSite	<code>SameSite=Lax</code>
Cookie sem flag Secure	<code>Secure</code> , condicional a produção
CSP não definido	<code>script-src 'none'</code> e origens restritas
Anti-clickjacking	<code>X-Frame-Options: DENY</code>
HSTS	<code>max-age=31536000; includeSubDomains</code>
X-Content-Type-Options	<code>nosniff</code>
Cache-control	<code>no-store</code> nas páginas de cliente logado

O achado que importava

Sem token CSRF e sem `SameSite`, um site malicioso visitado por um cliente logado poderia disparar um `POST` para `/checkout` usando o cookie dele — o navegador anexa o cookie automaticamente, e não sabe distinguir um formulário nosso de um de outro site. O pedido sairia de verdade.

Um benefício accidental do CSS puro. A loja não usa uma linha de JavaScript, o que permite a política mais restritiva possível — `script-src 'none'` — e torna XSS praticamente inviável: não há onde um script injetado executar.

O alerta que NÃO foi corrigido, e por quê. *Sub Resource Integrity* refere-se ao `<link>` do Google Fonts. SRI trava o hash do arquivo no HTML, e o `fonts.googleapis.com` **serve CSS diferente conforme o navegador** — o hash mudaria por visitante e a página quebraria para parte deles. A mitigação adotada é o próprio CSP, que restringe `style-src` e `font-src` a esses dois domínios. A correção definitiva seria hospedar as fontes junto com a aplicação.

Demais controles

- Senhas apenas como hash `scrypt` (`werkzeug.security`)
- `DATABASE_URL` e `SECRET_KEY` por variável de ambiente; `.env` no `.gitignore` e nunca versionado
- A aplicação conecta com o papel `torra_app`, sem `SUPERUSER` — verificado: `DROP TABLE` é negado
- Validação no formulário e no banco

10. Backup, deploy e limitações

Backup e restauração — testados

Backup só é confiável quando a restauração também é testada.

```
pg_dump -F c          → 23 KB
origem                → 12 produtos · 4 categorias · 2 clientes · 2 pedidos · 3 itens
restaurado em banco novo → contagens IDÊNTICAS
constraints no destino → 11 CHECK · 4 FK · 5 PK · 3 UNIQUE · 26 NOT NULL
índices no destino    → 4 de 4

-- e a regra ainda funciona no banco restaurado:
ERROR: new row for relation "produtos" violates check constraint "ck_produtos_preco"
```

Não voltou só a estrutura — a **regra de negócio** voltou ativa.

Um obstáculo que vale registrar. O `pg_dump` local era 17.5 e o servidor do Railway é 18.6 — o `pg_dump` se recusa a dumpar um servidor mais novo que ele, porque formatos internos mudam entre versões maiores. A solução foi rodar o dump de dentro do container do banco, onde as ferramentas são da versão exata.

Deploy

Aplicação	Railway · unicorn via <code>Procfile</code>
Banco	PostgreSQL 18.6 gerenciado, no mesmo projeto
Domínio	<code>nivaldo.felipefurlan.com.br</code> , HTTPS com Let's Encrypt
Versionamento	GitHub, 23 commits — um por etapa

O material sugere Render + Neon; optamos pelo Railway porque hospeda aplicação e banco no mesmo projeto, ligados por *reference variable*, e o critério de avaliação é agnóstico de ferramenta. A migração seria barata: o Render lê o mesmo `Procfile` e o Neon entrega a `DATABASE_URL` no mesmo formato, que o `app.py` já normaliza.

Pendência conhecida. O deploy automático a cada `push` não está ativo. O repositório pertence a outra conta do GitHub e o Railway respondeu de forma explícita ao tentar criar o gatilho: *"no one in the project has access to it"*. Depende de o dono do repositório autorizar a Railway GitHub App. Enquanto isso, o deploy é disparado manualmente e funciona normalmente.

Limitações do MVP, documentadas

- **Sem pagamento real** — o pedido nasce como `CRIADO`
- **Sem frete** — o total é a soma dos itens
- **Sem controle de lote e data de torra** — a evolução mais natural do domínio: café tem validade curta e a data de torra é informação de venda
- **Sem área administrativa** — o catálogo entra via `seed.sql`

- **Carrinho na sessão** — não sobrevive à troca de dispositivo. É exatamente onde um Redis passaria a fazer sentido

A lógica do projeto é cumulativa: boa análise → boa modelagem → bom banco → boa aplicação → boa qualidade → bom deploy. Cada etapa deste relatório depende da anterior, e é isso que o conjunto pretende demonstrar.